
Work Breakdown Structure

AI-Powered Analytics Dashboard

Date: March 10, 2026

Document Type: WBS / Project Planning

Status: Draft

This document presents the Work Breakdown Structure for an AI-powered analytics dashboard that enables users to upload spreadsheet data and query it using natural language. The WBS covers all phases from planning through deployment and handoff.

Table of Contents

- 1. Assumptions & Unknowns
- 2. Work Breakdown Structure
- 3. Effort Summary

1. Assumptions & Unknowns

Assumptions

- Data is in CSV/Excel format (spreadsheets), volume under 100 MB.
- Natural language querying uses an LLM API (e.g. OpenAI) rather than a custom-trained model.
- "Fast" means sub-3-second query response for typical questions.
- "Tight budget" implies a small team (1-2 developers), open-source stack, and cloud-managed LLM API.
- MVP scope: upload data, ask questions, see chart/table answers. No multi-tenant auth or enterprise SSO.
- Target deployment: single cloud instance or serverless (e.g. Vercel + managed DB).
- No mobile-native app; responsive web UI is sufficient.

Open Questions / Unknowns

- Exact data schemas and number of distinct spreadsheet types.
- Acceptable monthly cloud/API spend ceiling.
- Whether users need to save/share dashboards or just ad-hoc queries.
- Authentication requirements (open access vs. login-protected).
- Data sensitivity / compliance requirements (GDPR, HIPAA, etc.).
- Expected concurrent user count.

2. Work Breakdown Structure

1. Project Management

ID	Task	Description	Effort
1.1	Project Kickoff & Planning	Define scope, timeline, and roles. Align with client on MVP features.	2d
1.2	Requirements Gathering	Collect sample data, document question patterns, define acceptance criteria.	2d
1.3	Risk & Assumption Log	Document assumptions, unknowns, and mitigation plans.	0.5d

ID	Task	Description	Effort
1.4	Sprint Planning & Tracking	Ongoing backlog grooming, standups, retrospectives.	Ongoing
1.5	Client Demos & Feedback Loops	Bi-weekly demos to gather feedback and adjust priorities.	Ongoing

2. Data Ingestion & Storage

ID	Task	Description	Effort
2.1	File Upload Interface	Drag-and-drop UI for CSV/Excel uploads with validation and error feedback.	2d
2.2	Data Parsing & Normalization	Parse spreadsheets, infer column types, handle edge cases (dates, nulls).	3d
2.3	Database Schema Design	Design schema to store uploaded datasets; support multiple tables.	1d
2.4	Data Storage Layer	Implement storage using PostgreSQL or DuckDB for analytical queries.	2d
2.5	Data Preview & Metadata	Show column names, types, row counts, and sample rows after upload.	1d

3. Natural Language Query Engine

ID	Task	Description	Effort
3.1	LLM Integration Layer	Connect to OpenAI (or similar) API; manage keys, retries, rate limits.	2d
3.2	Schema-Aware Prompt Engineering	Build prompts that include table schema so the LLM generates accurate SQL.	3d
3.3	Text-to-SQL Pipeline	Convert natural language to SQL, execute against DB, return results.	3d
3.4	Query Validation & Safety	Sanitise generated SQL; prevent destructive operations; handle errors.	2d
3.5	Response Formatting	Return results as structured data (table rows, aggregates) for the frontend.	1d
3.6	Caching & Performance	Cache frequent queries; optimise for sub-3-second response times.	2d

4. Frontend / Dashboard UI

ID	Task	Description	Effort
4.1	UI Framework Setup	Scaffold React/Next.js project; configure Tailwind CSS and component library.	1d

ID	Task	Description	Effort
4.2	Data Upload Page	Page for uploading, listing, and managing datasets.	2d
4.3	Query Interface	Chat-style input bar for natural language questions with autocomplete hints.	3d
4.4	Results Display (Tables)	Render query results as sortable, paginated data tables.	2d
4.5	Results Display (Charts)	Auto-select chart type (bar, line, pie) based on result shape; use Chart.js or Recharts.	3d
4.6	Dashboard Layout & Navigation	Top nav, sidebar for datasets, responsive layout.	2d
4.7	Loading States & Error Handling	Skeleton loaders, friendly error messages, retry prompts.	1d

5. Backend API & Infrastructure

ID	Task	Description	Effort
5.1	API Framework Setup	Set up FastAPI (Python) or Next.js API routes; configure CORS.	1d
5.2	File Upload Endpoint	Handle multipart uploads, validate file types and size limits.	1d
5.3	Query Endpoint	Accept NL question, orchestrate LLM call, return results.	2d
5.4	Environment & Config Management	Manage API keys, DB credentials, and feature flags securely.	0.5d
5.5	Logging & Monitoring	Structured logging; basic health-check endpoint; error alerting.	1d

6. Testing & Quality Assurance

ID	Task	Description	Effort
6.1	Unit Tests (Backend)	Test parsing, SQL generation, response formatting.	2d
6.2	Integration Tests	End-to-end tests: upload file, ask question, verify response.	2d
6.3	Frontend Tests	Component tests and basic E2E tests with Playwright or Cypress.	2d
6.4	LLM Output Validation	Evaluate accuracy of generated SQL against a golden-question set.	2d
6.5	Performance Testing	Verify sub-3-second response for datasets up to 100 MB.	1d

7. Deployment & DevOps

ID	Task	Description	Effort
7.1	CI/CD Pipeline	GitHub Actions (or similar) for lint, test, build, deploy.	1d

ID	Task	Description	Effort
7.2	Infrastructure Provisioning	Set up cloud resources (Vercel/Railway + managed Postgres).	1d
7.3	Domain & SSL Setup	Configure custom domain, HTTPS, DNS.	0.5d
7.4	Production Deploy & Smoke Test	Deploy to production, verify all flows end-to-end.	1d

8. Documentation & Handoff

ID	Task	Description	Effort
8.1	User Guide	Short guide showing how to upload data and ask questions.	1d
8.2	Developer Documentation	README, architecture diagram, environment setup instructions.	1d
8.3	Client Handoff & Training	Walkthrough session with client; hand over credentials and repos.	0.5d

3. Effort Summary

Phase	Work Packages	Estimated Effort
1. Project Management	5	4.5 days + 2 ongoing
2. Data Ingestion & Storage	5	9.0 days
3. Natural Language Query Engine	6	13.0 days
4. Frontend / Dashboard UI	7	14.0 days
5. Backend API & Infrastructure	5	5.5 days
6. Testing & Quality Assurance	5	9.0 days
7. Deployment & DevOps	4	3.5 days
8. Documentation & Handoff	3	2.5 days
TOTAL	40	61.0 days + ongoing PM

Estimated Timeline

With a total of approximately **61 person-days** of discrete work plus ongoing project management, a single full-stack developer can deliver the MVP in roughly **14–17 weeks**, accounting for context-switching, client feedback cycles, and integration time. A two-person team could compress this to approximately **8–11 weeks**.

Budget-Conscious Stack Recommendation

- **Frontend:** Next.js + Tailwind CSS (free, open source)
- **Backend:** Next.js API routes or FastAPI (free, open source)
- **Database:** PostgreSQL on Supabase free tier or Railway (\$5/mo)
- **LLM API:** OpenAI gpt-4o-mini for cost efficiency (~\$0.15 per 1M input tokens)
- **Hosting:** Vercel free/hobby tier or Railway starter (\$5/mo)
- **Estimated monthly run cost:** \$10–50/mo depending on query volume

End of Document